CS & IT ENGINEERING

Operating System

Deadlock

Lecture -2

By- Vishvadeep Gothi sir

Recap of Previous Lecture

doubt ?

Topic Deadlock

Topic Deadlock Prevention

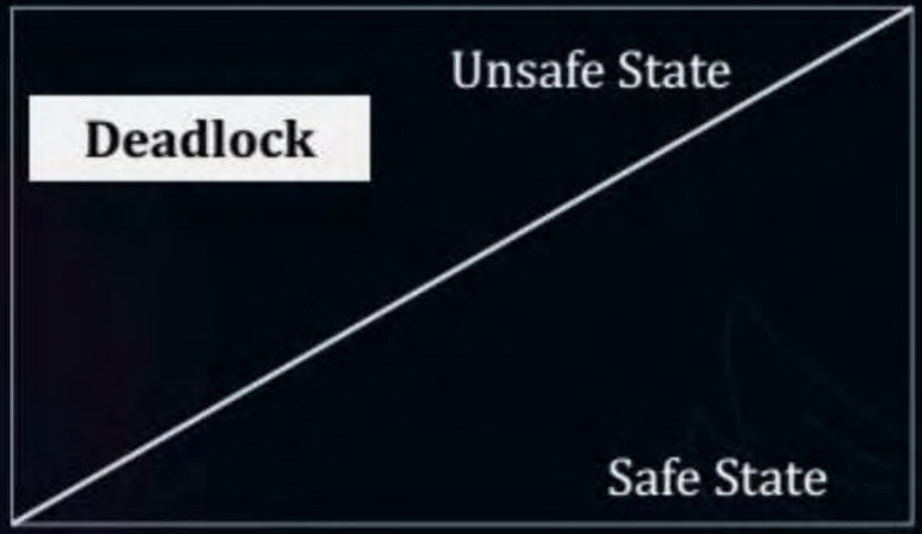
Topics to be Covered

Topic Deadlock Avoidance

Topic Banker's Algorithm

Topic : Deadlock Avoidance

In deadlock avoidance, the OS tries to keep system in safe state



Topic : Deadlock Avoidance

In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.

In deadlock avoidance every process will have to announce the max no. of resources needed for execution.

⇓

max

Topic : Banker’s Algorithm

The banker’s algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety

Topic : Banker's Algorithm

already allocated

free

Process	Allocation	Max	Available
P1	1	3	~~1~~
P2	5	8	After P3 = (4)
P3	3	4	After P1 = 5
P4	2	7	After P2 = 10
			After P4 = 12

Need = max − allocation

2 ✓

3 ✓

1 ✓

5 ✓

1. any process P_i, need$_{P_i}$ ≤ available

available = available + allocation$_{P_i}$

if all processes completed which means system is safe

safe sequence ⇒ < P3, P1, P2, P4 >

< P3, P4, P1, P2 >

⇓

safe or not ?

⇓

not safe

in last Question :-

safe or unsafe sequence

Available

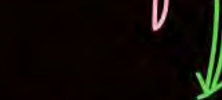
Safe ⇐ 1. <P3, P2, P4, P1>

Safe ⇐ 2. <P3, P1, P4, P2>

ex:- (2)

	Allocation	max	available	Need
P1	2	6	1	4
P2	1	2	After P2: 2	1 ✓
P3	3	7		4
P4	1	6		5

unsafe state



possibility of deadlock

Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	~~3 3 2~~	7 4 3
P_1	2 0 0	3 2 2	After P1: 5 3 2	1 2 2 ✓
P_2	3 0 2	9 0 2	After P3: 7 4 3	6 0 0
P_3	2 1 1	2 2 2	After P0: 7 5 3	0 1 1 ✓
P_4	0 0 2	4 3 3	After P2: 10 5 5	4 3 1
			After P4: 10 5 7	

→ system is safe

safe sequence => <P1, P3, P0, P2, P4>

Topic : Banker's Algorithm

1. Allocation:
2. Max:
3. Need:
4. Available:

Topic : Banker's Algorithm

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
   Initialize: Work = Available
   Finish[i] = false; for i=1, 2, 3, 4....n
2. Find an i such that both (a) Finish[i] = false
   (b) Needi <= Work
   if no such i exists goto step (4)
3. Work = Work + Allocation[i]
   Finish[i] = true goto step (2)
4. If Finish [i] = true for all i
   then the system is in a safe state

H.W. Safe or unsafe

#Q.

Process	Allocation A B C D	Max A B C D	Available A B C D
P1	0 0 1 2	0 0 1 2	1 5 2 0
P2	1 0 0 0	1 7 5 0	
P3	1 3 5 4	2 3 5 6	
P4	0 6 3 2	0 6 5 4	
P5	0 0 1 4	0 6 5 6	

Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	~~0 1 0~~ 1 1 2	7 5 3	~~3 3 2~~ 2 3 0	~~7 4 3~~ 6 4 1
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

[NAT]

#Q. What will happen if process P0 requests one additional instance of resource type A and two instances of resource type C ?

$Request_{P_0} = \langle 1, 0, 2 \rangle \longrightarrow$ unsafe $\Rightarrow$ Rejected

---

1. if $Request_i \leq Need_i$, goto step 2 otherwise invalid request
2. if $Request_i \leq Available$, goto step 3 otherwise request pending
3. $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   $Available = Available - Request_i$
4. Run safety algo. If safe $\Rightarrow$ request granted
   if unsafe $\Rightarrow$ Request rejected

Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	~~3 3 2~~ 3 2 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2	safe	6 0 0
P_3	~~2 1 1~~ 3 2 1	2 2 2		~~0 1 1~~ 0 0 1
P_4	0 0 2	4 3 3		4 3 1

[NAT]

#Q. What will happen if process P3 requests one additional instance of resource type B ?

<0, 1, 0>

⇓

granted

Topic : Resource Request Algorithm

1. If $Request_i$ <= $Need_i$
   Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If $Request_i$ <= Available
   Goto step (3); otherwise, P_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:
   Available = Available – Requesti
   $Allocation_i$ = $Allocation_i$ + $Request_i$
   $Need_i$ = $Need_i$– $Request_i$

Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

[NAT]

H.W.

#Q. What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C ?

What will happen if process P3 requests one additional instance of resource type B?

2 mins Summary

Topic Deadlock Avoidance

Topic Banker’s Algorithm

Happy Learning

THANK - YOU